DBMS Assignment

Real-World Database Schema: QuickMart (Quick Commerce)

Application Overview

Name: QuickMart (Inspired by Blinkit / Quick Commerce)

Purpose: QuickMart is a 10-minute grocery and essential delivery platform operating on a "dark store" model. Unlike traditional e-commerce platforms that ship from centralized warehouses over several days, this application routes customer orders to hyper-local micro-fulfillment centers (dark stores) based on the customer's location. The database handles high-speed inventory tracking across multiple store locations, maps customers to their nearest store, manages real-time order states, and assigns delivery agents.

Required Tables and Their Purpose

- **Users:** Stores core customer profile information and authentication details.
- **User_Addresses:** Manages multiple delivery locations (Home, Work, etc.) for a single user, including geo-coordinates.
- **Dark_Stores:** Represents the local fulfillment centers. Determines which store serves which geographical radius.
- **Categories:** Organizes products into a hierarchy (e.g., Groceries > Dairy > Milk) for app navigation.
- **Products:** The global master catalog of all items available on the platform, holding static details.
- **Store_Inventory:** A junction table that maps products to specific dark stores, tracking the exact real-time stock and localized selling price.
- **Orders:** The central transactional table tracking the lifecycle of a purchase from placement to delivery.
- **Order_Items:** Stores the individual products, quantities, and locked-in prices for a specific order.
- **Delivery_Agents:** Manages the fleet of riders, their current status, and location.
- **Payments:** Tracks transaction statuses, amounts, and payment methods.

Detailed Table Schema

1. Users

Column Name	Data Type	Key / Constraint	Description
UserID	INT	Primary Key	Unique identifier for the customer
FullName	VARCHAR	Not Null	Customer's full name
PhoneNumber	VARCHAR	Unique, Not Null	Primary contact number
Email	VARCHAR	Unique	Customer's email address
CreatedAt	DATETIME	Not Null	Timestamp of account creation

2. User_Addresses

Column Name	Data Type	Key / Constraint	Description
AddressID	INT	Primary Key	Unique identifier for the address
UserID	INT	Foreign Key	Links to Users table
AddressLine	VARCHAR	Not Null	Flat, building, street
Pincode	VARCHAR	Not Null	Postal code
Latitude	DECIMAL	Not Null	GPS coordinate for delivery routing
Longitude	DECIMAL	Not Null	GPS coordinate for delivery routing
Label	VARCHAR	None	E.g., "Home", "Office"

3. Dark_Stores

Column Name	Data Type	Key / Constraint	Description
StoreID	INT	Primary Key	Unique identifier for the local store
StoreName	VARCHAR	Not Null	Internal name of the location
Pincode	VARCHAR	Not Null	Postal code the store operates in
Latitude	DECIMAL	Not Null	Store GPS coordinate
Longitude	DECIMAL	Not Null	Store GPS coordinate
IsActive	BOOLEAN	Default True	Whether the store is currently accepting orders

4. Categories

Column Name	Data Type	Key / Constraint	Description
CategoryID	INT	Primary Key	Unique identifier for the category
Name	VARCHAR	Not Null	E.g., "Dairy & Bakery", "Snacks"
ParentCategoryID	INT	Foreign Key	Self-references Categories for sub-categories

5. Products

Column Name	Data Type	Key / Constraint	Description
ProductID	INT	Primary Key	Unique identifier for the item
CategoryID	INT	Foreign Key	Links to Categories table
Name	VARCHAR	Not Null	Name of the product
Description	TEXT	None	Product details, ingredients, etc.
MRP	DECIMAL	Not Null	Maximum Retail Price
ImageURL	VARCHAR	None	Link to product thumbnail

6. Store_Inventory

Column Name	Data Type	Key / Constraint	Description
InventoryID	INT	Primary Key	Unique identifier for the inventory record
StoreID	INT	Foreign Key	Links to Dark_Stores
ProductID	INT	Foreign Key	Links to Products
StockQuantity	INT	Not Null, >= 0	Current units available at this exact store
SellingPrice	DECIMAL	Not Null	Store-specific discounted price

7. Orders

Column Name	Data Type	Key / Constraint	Description
OrderID	INT	Primary Key	Unique identifier for the order
UserID	INT	Foreign Key	Links to Users
StoreID	INT	Foreign Key	Links to Dark_Stores
AddressID	INT	Foreign Key	Links to User_Addresses
AgentID	INT	Foreign Key (Null)	Links to Delivery_Agents (assigned later)
TotalAmount	DECIMAL	Not Null	Final billable amount
OrderStatus	VARCHAR	Not Null	Placed, Packed, OutForDelivery, Delivered
OrderTime	DATETIME	Not Null	Timestamp of order placement

8. Order_Items

Column Name	Data Type	Key / Constraint	Description
OrderItemID	INT	Primary Key	Unique identifier for the line item
OrderID	INT	Foreign Key	Links to Orders
ProductID	INT	Foreign Key	Links to Products
Quantity	INT	Not Null, > 0	Number of units purchased
PriceAtPurchase	DECIMAL	Not Null	Locked-in price at the time of checkout

9. Delivery_Agents

Column Name	Data Type	Key / Constraint	Description
AgentID	INT	Primary Key	Unique identifier for the rider
Name	VARCHAR	Not Null	Agent's full name
PhoneNumber	VARCHAR	Unique, Not Null	Agent's contact number
VehicleType	VARCHAR	Not Null	E.g., "Bike", "EV", "Cycle"
CurrentStatus	VARCHAR	Not Null	Available, OnTrip, Offline

10. Payments

Column Name	Data Type	Key / Constraint	Description
PaymentID	INT	Primary Key	Unique identifier for the transaction
OrderID	INT	Foreign Key	Links to Orders
AmountPaid	DECIMAL	Not Null	Exact amount processed
Method	VARCHAR	Not Null	UPI, Credit Card, Cash on Delivery
PaymentStatus	VARCHAR	Not Null	Success, Pending, Failed, Refunded

Entity Relationships

One-to-Many (1:N)

- **User → User_Addresses:** One user can have multiple saved addresses, but an address record belongs to one user.
- **User → Orders:** A user can place multiple orders over time.
- **Category → Products:** A category contains multiple products.
- **Order → Order_Items:** A single order contains multiple items.
- **Dark_Store → Orders:** One store processes multiple orders in its area.
- **Delivery_Agent → Orders:** An agent can deliver multiple orders over their shift.
- **Order → Payments:** While usually 1:1, a single order might have multiple payment attempts.

Many-to-Many (M:N) resolved via Junction Tables

- **Dark_Stores ↔ Products:** Solved by the **Store_Inventory** table. A store holds many products, and a product exists in many stores. This is critical for quick commerce, as stock is decentralized.

- **Orders ↔ Products:** Solved by the **Order_Items** table. An order has many products, and a product can be in many different orders.